

ユニット 6・第8-9週

シグナルと共有メモリ

📅 第8-9週・日程は未定 📄 演習 TD5（解答 + 生成AIチャレンジ） 🎞 アニメーション 3 本

マニュアル M6・ユニットの理論

シグナルと共有メモリ

ユニット 6 の基本の読み物（約 1.5 時間）。出力はすべて実物で、コースのマシンで TD5 のプログラムを実行したものです。

目次

- | | | | |
|----|---------------|----|-----------------------------------|
| §1 | シグナル：送信、保留、配送 | §3 | <code>mmap</code> ：ファイルをメモリにマップする |
| §2 | ハンドラと競合のない同期 | §4 | 共有メモリとその代償 |

§1 シグナル：送信、保留、配送

シグナルはプロセスへの非同期の通知です。カーネル（不正なアクセスに SIGSEGV、読み手のいないパイプへの書き込みに SIGPIPE、子の終了に SIGCHLD）、端末（Ctrl-C → SIGINT、Ctrl-\ → SIGQUIT、Ctrl-Z → SIGTSTP）、または `kill(pid, sig)` を使う別のプロセスが送ります。シグナルごとに、プロセスは**動作**を持ちます：既定の動作（終了、core 付き終了、停止、再開、無視）、無視（SIG_IGN）、または**ハンドラ**関数の実行です。捕捉も無視もブロックもできないのは SIGKILL と SIGSTOP だけです。

`kill` はシグナルを**保留**の印を付けるだけです。カーネルは、プロセスがユーザモードに戻るとき（システムコールが終わるときや、コンテキストスイッチの後に CPU を得るとき）にそれを**配送**します。ただし、プロセスのマスクで**ブロック**されていれば待たされます。標準シグナルでは「保留」は 1 ビットです。ブロック中に同じシグナルが 3 つ届いても、ブロックを解くと 1 つしか配送されません。リアルタイムシグナル（SIGRTMIN ...）はキューに入り、値を運べます。

実践 → A6.1 シグナルの配送

§2 ハンドラと競合のない同期

ハンドラは `sigaction` で設定します（`signal()` はシステムによって意味が違います）。ハンドラの実行中は、同じシグナル（と `sa_mask` のシグナル）がブロックされたままです。SA_RESTART があれば中断されたシステムコールは自動で再開し、なければ EINTR で失敗します（ユニット 2）。ハンドラはプログラムをどこでも中断するので、使えるのは**非同期シグナル安全な関数**（`write` は可、`printf` と `malloc` は不可）だけで、プログラムとのやり取りは `volatile sig_atomic_t` の変数を通して行います。

`pause()` でシグナルを待つ方法には**競合**があります。最後の確認と `pause` の呼び出しの間にシグナルが届くと、ハンドラが先に実行され、`pause` は永遠に待ちます。解決策は、`sigprocmask` でシグナルを**ブロック**し、条件を確かめ、`sigsuspend(mask)` で待つことです。`sigsuspend` はマスクの置き換えと

睡眠を1つのアトミックな操作で行います。「早く」届いたシグナルは保留されたままで、`sigsuspend` の中で配送されます。

C tds/td5/mysignal.c

```

/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores the default
action
 * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT <pid>, kill
-KILL <pid>
 * After the n-th SIGUSR1 the default action is back: the next SIGUSR1 terminates the
process.
 */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be interrupted mid-
buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n', 0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 || write_all(STDERR_FILENO,
digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);

    sa.sa_handler = on_usr1;
    sigemptyset(&sa.sa_mask);          /* nothing extra blocked while the handler runs

```

```
*/
sa.sa_flags = 0;
exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

/* Block SIGUSR1 outside sigsuspend: a signal arriving between the test of
`received`
and the wait stays pending instead of being lost (the pause() race). */
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}
```

実際の出力

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P; kill -USR1
$P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status: $?"
(128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

ほかの道具： `alarm(s)` は `SIGALRM` を予約します。 `glibc` の `sleep` は（ `alarm` ではなく）`clock_nanosleep` を使います。 `signalfd` はシグナルをディスクリプタから読むデータに変え、`pidfd_send_signal` はすでに再利用された PID にシグナルを送ってしまうのを防ぎます。

実践 → TD5・1と2

§3 mmap：ファイルをメモリにマップする

`mmap(addr, len, prot, flags, fd, offset)` はアドレスの範囲を確保し、それをファイルに（`MAP_ANONYMOUS` なら何にも）結び付けます。何もコピーしません。各ページは最初のアクセス時に

ページフォールトで読み込まれます（ユニット 4）。`MAP_SHARED` では書き込みはファイルのページキャッシュに入り、同じファイルをマップしている他のプロセスにも見え、ディスクにも届きます（`msync` かマッピング解除のとき）。`MAP_PRIVATE` では最初書き込みでページがコピーされ、ファイルは変わりません。`len` はページ単位に切り上げられ、`offset` はページサイズの倍数でなければなりません。

最後のページの中でファイルの終わりより先にアクセスすると、保存されないゼロが返ります。裏にファイルのないページ全体に触れると **SIGBUS** になります（アドレスは有効ですが、読み込むデータがありません）。マップしたファイルを大きくするには、先に `ftruncate` を呼びます。`PROT_*` の保護はページ単位で、x86 では書けることは読めることを意味します：

実際の出力

```
$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes
```

実際の出力

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:   *p = 0
write:  *p = 42
```

実際の出力

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

実践 → A6.3 mmap の限界・TD5・3 と 4

§4 共有メモリとその代償

メモリの共有とは、2つのプロセスの仮想ページが同じ物理フレームを指すようにすることです。

待ち合わせ場所とインターフェースが必要です：共通のファイルを `MAP_SHARED` で `mmap` する。POSIX の `shm_open` + `mmap`（`/dev/shm` 内の名前、今の推奨）。または System V：`ftok(file, id)` がキーを作り、`shmget` がセグメントを作るか見つけ、`shmat` が取り付け、`shmdt` が外し、`shmctl(IPC_RMID)` が破棄します。セグメントは誰かが破棄するまでプロセスより長く残ります：

実際の出力

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

規則は 2 つです。1 つ目：各プロセスはその領域を**別のアドレス**で見るので、ポインタは保存しません（オフセットを保存します）。2 つ目：共有メモリはカーネルを通らないので最速の通信ですが、まさにそのために**何も同期してくれません**。複数のプロセスが同じカウンタに「読む、1 足す、書く」を行うと更新が失われます。プロセッサのアトミック操作（`__atomic_add_fetch`、x86 では `lock xadd`）を使えば失われません：

実際の出力

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

実際の出力

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568 357571 357569
357565 357574 357573 357570 357576 357578 357575 357577 357579
```

実践 → A6.2 共有メモリ・TD5・5